Step 1 — One Specialist's Drafting Logic, Working Alone, No Agent Behavior — Solution

Back to this step · Hint 1 · Hint 2 · Solution

Story — Step 1: a team is only as good as each member. Start with one specialist — the Writer, because it needs no tools — and prove its core skill alone: findings in, article out, as one LCEL chain. **If not:** in Step 5, a bad article could come from the Writer, the Reviewer, the routing or the state, and you'd be debugging all four at once.

After this step the project runs: `python main.py` writes three test articles from made-up findings, and the test files print `OK`.

The shared files

These are the same files as Projects 1-3 — Doc01's config, errors and logger, and Project 1's prompt loader — with the package folder `src`, and the prompts folder changed to `src/prompts`. Project 3's `get_model()` comes with them.

Story — `exceptions.py`, `config.py`, `utils/logger.py`, `utils/prompts.py`: settings load and fail loudly at startup, logging is set up once, prompts live in `.txt` files. **If not:** each agent would repeat this setup its own way.

```
# src/exceptions.py
# The project's named errors. MissingConfigError: a required setting is
# missing from .env. (Doc01's file.)
class MissingConfigError(Exception):
    """Raised when a required setting is missing from the environment."""
    pass
```

```
# src/config.py
# Loads the settings from .env into a checked Config object. load_config()
# raises MissingConfigError when OPENAI_API_KEY is missing. (Doc01's file.)
import os
from dataclasses import dataclass
from dotenv import load_dotenv
from src.exceptions import MissingConfigError
```

```
@dataclass
class Config:
    openai_api_key: str
    log_level: str

def require_env(key):
    value = os.getenv(key)
    if value is None or value == "":
        raise MissingConfigError(
            f"Required environment variable is missing: {key}"
        )
    return value

def load_config():
    load_dotenv()
    api_key = require_env("OPENAI_API_KEY")
    log_level = os.getenv("LOG_LEVEL", "INFO")
    return Config(openai_api_key=api_key, log_level=log_level)
```

```
# src/utils/logger.py
# get_logger(name): one logging setup for every file — a timestamp, the
# file name and the level on every line. (Doc01's file.)
import logging
import os

def get_logger(name):
    logger = logging.getLogger(name)

    if not logger.handlers:
        level_name = os.getenv("LOG_LEVEL", "INFO")
        level = getattr(logging, level_name.upper(), logging.INFO)

        handler = logging.StreamHandler()
        handler.setLevel(level)
        log_format = "%(asctime)s %(name)s %(levelname)s %(message)s"
        formatter = logging.Formatter(log_format)
        handler.setFormatter(formatter)

        logger.addHandler(handler)
        logger.setLevel(level)

    return logger
```

```
# src/utils/prompts.py
# load_prompt(name): reads src/prompts/<name>.txt, so prompts live in text
```

```
# files instead of Python strings. (Project 1's file.)

def load_prompt(name):
    path = "src/prompts" + "/" + name + ".txt"
    with open(path) as prompt_file:
        return prompt_file.read().strip()
```

Also create empty `__init__.py` files in `src/utils/`, `src/models/` and `src/agents/`.

Back to this step · Hint 1 · Hint 2 · Solution

Solution — a reusable Writer with strict rules, tested on 3 inputs

Story — **`models/llm.py`** **:** one place builds the model for every agent — the same `get_model()` as Project 3, since Step 5's graph nodes only receive the state. **If not:** four agents would build four models with four settings.

```
# src/models/llm.py
# get_model(): the one ChatOpenAI every agent uses. ChatOpenAI reads
# OPENAI_API_KEY from the environment, filled by load_config().
from langchain_openai import ChatOpenAI

def get_model():
    return ChatOpenAI(model="gpt-4o-mini", temperature=0)
```

Story — **`prompts/writer.txt`** **:** the Writer's rules: a titled 150-250-word article, **only** the given findings, and keep each fact's `[source]` tag. **If not:** the article could quietly mix in facts nobody researched.

`src/prompts/writer.txt`

```
You are the Writer on a content team. Write a short article (150-250
words) with a title, using ONLY the findings below. Do not add facts
that are not in the findings. Keep each fact's [source] tag.

Findings:
{findings}
```

(Step 2 adds a feedback section to this prompt.)

Story — **`agents/writer_agent.py`** **:** `build_writer_chain()` is the Writer's whole skill; `run_writer()` is the one function other code calls. **If not:** the chain would live inside `main.py`, and Step 2 couldn't wrap it.

```
# src/agents/writer_agent.py
# The Writer agent. build_writer_chain() is the LCEL chain (Doc05);
# run_writer(findings) is the one function other code calls.
from langchain_core.output_parsers import StrOutputParser
from langchain_core.prompts import ChatPromptTemplate
```

```
from src.models.llm import get_model
from src.utils.prompts import load_prompt

def build_writer_chain():
    prompt = ChatPromptTemplate.from_template(load_prompt("writer"))
    return prompt | get_model() | StrOutputParser()

def run_writer(findings):
    chain = build_writer_chain()
    return chain.invoke({"findings": findings})
```

Story — `main.py` **:** three test inputs, including a nearly empty one **on purpose** — to see now what the Writer does with almost nothing, before Research ever feeds it something thin. **If not:** you'd first see that behavior in the full pipeline, with four agents to suspect.

```
# main.py
# The file you run: python main.py
# Step 1 version: runs the Writer on three made-up findings and prints
# each article. Later steps replace this with the full team.
import sys

from src.agents.writer_agent import run_writer
from src.config import load_config
from src.exceptions import MissingConfigError

TEST_FINDINGS = [
    "Range:\n- A typical e-bike battery gives 40 to 100 km per charge. "
    "[battery-review]\n- Most e-bikes are pedal-assist. [bike-basics]",
    "Savings:\n- Remote workers save 72 minutes a day of commuting. "
    "[commute-study]\n- Loneliness is the main downside. [wellbeing]",
    "- Solar panels exist.",
]

def main():
    try:
        load_config()
    except MissingConfigError as e:
        print(f"Setup problem: {e}")
        sys.exit(1)

    for findings in TEST_FINDINGS:
        print("=" * 60)
        print(run_writer(findings))

if __name__ == "__main__":
    main()
```

Run it: `python main.py`, and **read** all three articles. For the thin input, a good Writer writes something very short, or says there's too little to go on — it shouldn't invent facts about solar panels. If it invents, tighten the "ONLY the findings" wording now.

Back to this step · Hint 1 · Hint 2 · Solution

The Step 1 tests

Story — `tests/unit/test_config.py`: the same three checks as Doc01's `test_config.py` — a missing setting, an empty one, and a config that loads — with no API call. It tests `require_env()` with a made-up setting name, so your real `.env` is never touched. **If not:** a broken `.env` would first show up as a confusing OpenAI error.

```
# tests/unit/test_config.py
# Checks the settings code with no API call. Each case prints OK or WRONG.
# It only changes settings inside this test run — your .env file is never
# touched.
# Run from the repo root: python -m tests.unit.test_config
import os

from src.config import load_config, require_env
from src.exceptions import MissingConfigError

TEST_SETTING = "CONTENTFORGE_TEST_SETTING"

def test_missing_setting():
    os.environ.pop(TEST_SETTING, None)
    result = "WRONG"
    try:
        require_env(TEST_SETTING)
    except MissingConfigError:
        result = "OK"
    print(f"case 1 (missing setting) -> {result}")

def test_empty_setting():
    os.environ[TEST_SETTING] = ""
    result = "WRONG"
    try:
        require_env(TEST_SETTING)
    except MissingConfigError:
        result = "OK"
    print(f"case 2 (empty setting)   -> {result}")

def test_config_loads():
    os.environ["OPENAI_API_KEY"] = "sk-test-123"
    config = load_config()
    result = "OK"
```

```
    if config.openai_api_key != "sk-test-123":
        result = "WRONG"
    print(f"case 3 (config loads)    -> {result}")

if __name__ == "__main__":
    test_missing_setting()
    test_empty_setting()
    test_config_loads()
```

Story — `tests/integration/test_live_agents.py`: the checks that need the real model. It starts with one case — the Writer turns findings into an article that keeps its `[source]` tag — and every later step adds its own cases to this same file. **If not:** "the Writer works" would mean "it worked the one time I tried it".

```
# tests/integration/test_live_agents.py
# Runs the agents against the real OpenAI API — each step adds cases.
# Each case prints OK or WRONG. Needs a valid OPENAI_API_KEY in .env.
# Run from the repo root: python -m tests.integration.test_live_agents
from src.agents.writer_agent import run_writer
from src.config import load_config

FINDINGS = (
    "Range:\n- A battery gives 40 to 100 km per charge. [battery-review]\n"
    "Rules:\n- EU e-bike motors are limited to 25 km/h. [eu-rules]\n"
)

def test_writer_keeps_sources():
    draft = run_writer(FINDINGS)
    result = "OK"
    if "[" not in draft:
        result = "WRONG"
    print(f"case 1 (writer keeps sources) -> {result}")

if __name__ == "__main__":
    load_config()
    test_writer_keeps_sources()
```

Create the empty folders `tests/unit/` and `tests/integration/` too. Tests run as modules from the repo root, so they need no `__init__.py`.

Every test prints `OK` or `WRONG` — you never have to judge the output yourself. `WRONG` means: read that test function and the code it calls.

Run the tests (from the repo root):

```
python -m tests.unit.test_config
python -m tests.integration.test_live_agents
```

Expected output:

```
case 1 (missing setting) -> OK
case 2 (empty setting)   -> OK
case 3 (config loads)    -> OK
```

```
case 1 (writer keeps sources) -> OK
```